

Lab Sheet 3 – OOP Java

Topic Focus: Constructors, Method Overloading, Encapsulation

Total Experiments: 5

Experiment 1 – Parameterized Constructor

Problem Description:

Create a `Book` class with attributes: `title`, `author`, and `price`.

Write a parameterized constructor to initialize these values and a method `displayDetails()` to print book details.

Input Format:

- Title (String)
- Author (String)
- Price (double)

Output Format:

- Details of the book in a formatted manner

Constraints:

- Price should be positive
- Strings should not be null

Method Signature:

```
public Book(String title, String author, double price)
public void displayDetails()
```

Example:

Input:

```
Java Basics
John Doe
450.50
```

Output:

```
Title: Java Basics
Author: John Doe
Price: 450.5
```

Explanation:

The constructor initializes the book details and the `displayDetails()` method prints them.

Approach:

- Create class with instance variables
 - Implement parameterized constructor
 - Implement method to display details
-

Experiment 2 – Default & Parameterized Constructors

Problem Description:

Create a `Car` class with attributes: `brand` and `model`. Implement both default and parameterized constructors. Default constructor should assign "Unknown" to `brand` and `model`.

Input Format:

- Brand (String)
- Model (String)

Output Format:

- Car brand and model

Constraints:

- Both `brand` and `model` should be strings

Method Signature:

```
public Car()  
public Car(String brand, String model)  
public void display()
```

Example:

Input:

```
Toyota  
Fortuner
```

Output:

```
Brand: Toyota  
Model: Fortuner
```

Explanation:

This shows constructor overloading.

Experiment 3 – Method Overloading

Problem Description:

Create a `MathOperation` class with overloaded methods `add()` to handle:

- Two integers
- Three integers
- Two double values

Input Format:

- Different sets of integers/doubles

Output Format:

- Sum of provided values

Constraints:

- No negative numbers for integers in this example

Method Signature:

```
public int add(int a, int b)
public int add(int a, int b, int c)
public double add(double a, double b)
```

Example:**Input:**

```
add(5, 10) → 15
add(5, 10, 15) → 30
add(5.5, 4.5) → 10.0
```

Explanation:

Method overloading allows same method name but different parameter lists.

Experiment 4 – Encapsulation with Getters/Setters

Problem Description:

Create a `Student` class with private attributes: `name` and `marks`. Use getter and setter methods to access and update values.

Input Format:

- Student name (String)
- Marks (int)

Output Format:

- Name and marks

Constraints:

- Marks should be between 0 and 100

Method Signature:

```
public String getName()
public void setName(String name)
public int getMarks()
public void setMarks(int marks)
```

Example:**Input:**

```
Name: Aman
Marks: 85
```

Output:

Name: Aman
Marks: 85

Explanation:

Encapsulation hides data using private variables and allows controlled access via methods.

Experiment 5 – Combining Constructors & Encapsulation

Problem Description:

Create an `Employee` class with private fields: `id`, `name`, and `salary`. Initialize them using a constructor and provide getter methods.

Input Format:

- Employee ID (int)
- Name (String)
- Salary (double)

Output Format:

- Employee details

Constraints:

- Salary must be positive
- ID must be greater than 0

Method Signature:

```
public Employee(int id, String name, double salary)
public int getId()
public String getName()
public double getSalary()
```

Example:

Input:

```
101
Ravi Kumar
55000
```

Output:

```
ID: 101
Name: Ravi Kumar
Salary: 55000.0
```

Explanation:

Demonstrates constructor usage along with encapsulation.
